Software Design Specifications

for

Sustainable Gardening Companion (GardenMate) v1.0

Prepared by:

Team 59 —
V. Nikhil Varma - SE23UARI131,
Srijan Chopra- SE23UARI118,
Rohit Bharadwaj - SE23UARI119,
Susmit Reddy - SE23UARI122,
Rishikesh - SE23UARI104

Software Engineering, IT BLOCK
2 GF — Mahindra University

Document Information

Title:	**Software Design Specification — Sustainable Gardening Companion (GardenMate) v1.0**
Project Manager:	**Document Version No: 1.0**
	Document Version Date: April 2026
Prepared By: Team 59 — V. Nikhil Varma, Rohit bharadwaj, Srijan Chopra, Susmit Reddy, Rishikesh Bheemani	**Preparation Date: April 2026**

Version History

Ver. No.	Ver. Date	Revised By	Description	Filename

Table of Contents

1 INTRODUCTION 4

1.1 PURPOSE 4
1.2 SCOPE 4
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS 4
1.4 REFERENCES 4

2 USE CASE VIEW 4

2.1 USE CASE 4

3 DESIGN OVERVIEW 4

3.1 DESIGN GOALS AND CONSTRAINTS 5
3.2 DESIGN ASSUMPTIONS 5
3.3 SIGNIFICANT DESIGN PACKAGES 5
3.4 DEPENDENT EXTERNAL INTERFACES 5
3.5 IMPLEMENTED APPLICATION EXTERNAL INTERFACES 5

4 LOGICAL VIEW 5

4.1 DESIGN MODEL 6
4.2 USE CASE REALIZATION 6

5 DATA VIEW 6

5.1 DOMAIN MODEL 6
5.2 D ATA MODEL (PERSISTENT DATA VIEW) 6
5.2.1 Data Dictionary 6

6 EXCEPTION HANDLING 6

7 CONFIGURABLE PARAMETERS 6

8 QUALITY OF SERVICE 7

8.1 AVAILABILITY 7
8.2 SECURITY AND AUTHORIZATION 7
8.3 LOAD AND PERFORMANCE IMPLICATIONS 7
8.4 MONITORING AND CONTROL 7

1 Introduction

This Software Design Specification (SDS) defines the detailed design of GardenMate (Sustainable Gardening Companion) v1.0, a cross-platform mobile application. It translates the requirements stated in the SRS v1.0 into a concrete, implementable design using the COMET methodology and UML notation, covering all five subsystems: User and Profile Management (SS1), Gardening Guide and Recommendations (SS2), Weather and Smart Alerts (SS3), Reminders and Notifications (SS4), and Community Forum and Moderation (SS5).

1.1 Purpose

This SDS defines the architectural and detailed design of all five GardenMate subsystems and is intended for use by the development team implementing each subsystem, QA personnel designing test plans, the academic instructor and teaching assistant evaluating the project, and project leads tracking design completeness against the SRS. The document provides sufficient design detail to enable direct implementation and unit-level testing.

1.2 Scope

This SDS applies to GardenMate v1.0. It covers the five functional subsystems derived from the 28 functional requirements (F1.1-F5.6) and 16 use cases (UC-1.1 through UC-5.4) defined in the SRS. Out of scope: server infrastructure provisioning, app store submission, and the internal design of third-party APIs (OpenWeatherMap, FCM/APNs, SendGrid, Twilio).

1.3 Definitions, Acronyms, and Abbreviations

API: Application Programming Interface. APNs: Apple Push Notification service. bcrypt: Password hashing algorithm (cost factor >= 12). COMET: Component-based Object Modelling and Engineering Technique (Gomaa, 2011). DTO: Data Transfer Object. FCM: Firebase Cloud Messaging. IEEE: Institute of Electrical and Electronics Engineers. JWT: JSON Web Token. NFR: Non-Functional Requirement. OTP: One-Time Password (6-digit, single-use, 5-minute TTL). REST: Representational State Transfer. SDS: Software Design Specification. SRS: Software Requirements Specification. SS1-SS5: Subsystems 1 through 5. TLS: Transport Layer Security (minimum v1.2). UC: Use Case. UML: Unified Modelling Language.

1.4 References

GardenMate Software Requirements Specification v1.0, Team 59, Mahindra University, March 2026. Gomaa, H. (2011). Software Modeling and Design: UML, Use Cases, Patterns, and Software Architectures. Cambridge University Press. IEEE Std 1016-2009 - IEEE Standard for Software Design Descriptions. IEEE Std 830-1998 - IEEE Recommended Practice for SRS. OpenWeatherMap API Documentation: https://openweathermap.org/api. Firebase Cloud Messaging: https://firebase.google.com/docs/cloud-messaging.

2 Use Case View

This section details the use cases that represent the most significant, central functionality of GardenMate - those with large design coverage that exercise many design elements across multiple subsystems. Each use case below is realized in the detailed design in Section 4.

2.1 Use Case

The following use cases represent key design-significant scenarios for GardenMate, selected because they exercise the most design elements and stress the most delicate design points.

Field	Description
Use Case ID	UC-1.1
Name	Register
Actors	User (Gardener), Email/SMS OTP Service
Preconditions	User has not previously registered. Network connectivity available.
Trigger	User selects Register on the application launch screen.
Basic Flow	1. User enters name, email/phone, and password. 2. System validates input format.

Basic Flow	3. System hashes password with bcrypt (cost 12) and creates a pending UserAccount. 4. System generates a 6-digit OTP (TTL=5 min) and sends it via the Email/SMS OTP Service. 5. User enters OTP. 6. System validates OTP. 7. System marks account as active, issues JWT, and redirects to Dashboard.
Alternative Flow	Step 2: Invalid format - system shows field-specific errors. Step 5: OTP delivery failure - user offered resend after 30 seconds. Step 7: Wrong OTP - system shows error; account locked after 3 consecutive failures.
Post Conditions	Verified UserAccount persisted. JWT issued. User directed to Dashboard.
Exceptions	Email/phone already registered: system shows error and offers Login or Password Reset. Network timeout: operation fails gracefully with retry prompt.
Includes	Verify OTP (UC-1.3)
Extends	---

Field	Description
Use Case ID	UC-2.1
Name	Get Planting Recommendations
Actors	User (Gardener), Weather API
Preconditions	User authenticated. Location and preferences configured. Weather API accessible.
Trigger	User navigates to Recommendations screen or taps Get Recommendations.
Basic Flow	1. System retrieves user location, soil type, and preferences. 2. System calls Weather API for current conditions and 5-day forecast. 3. RecommendationEngine correlates weather data, soil type, and preferences against crop database. 4. System returns ranked list of recommended crops with sow dates and care tips.
Alternative Flow	Step 2: Weather API unavailable - system uses cached data (max 6-hour cache) with staleness warning.
Post Conditions	Recommendation list displayed. Recommendation record persisted for history.
Exceptions	API unavailable, no cache: system proceeds with soil/preference analysis only; weather-dependent fields shown as unavailable.
Includes	Fetch Weather Data, Analyse Soil Type, Analyse User Preferences
Extends	---

Field	Description
Use Case ID	UC-3.2
Name	Get Weather-Based Alerts
Actors	User (Gardener), Weather API
Preconditions	User authenticated. Location configured. Weather API accessible.
Trigger	Dashboard loads or scheduler triggers weather refresh (default 30-minute interval).
Basic Flow	1. System fetches current and forecast data from Weather API. 2. RiskAssessor evaluates thresholds: frost (<3C), heatwave (>38C), heavy rain (>25mm/h), wind (>60km/h). 3. For each threshold exceeded, system generates a WeatherAlert. 4. ActionAdvisor maps each alert type to protective gardening actions. 5. Alerts displayed on Dashboard; push notification sent if user is not active.
Alternative Flow	No thresholds exceeded: all-clear status displayed on weather widget.
Post Conditions	WeatherAlert records created. Actions displayed. Notification dispatched if warranted.
Exceptions	API unavailable: cached data used with staleness flag. If no cache, graceful degradation with weather-unavailable indicator.
Includes	Fetch Weather Data, Detect Risk Level, Suggest Protective Action
Extends	Frost Alert, Heatwave Alert, Heavy Rain Alert, Wind Alert

Field	Description
Use Case ID	UC-4.1
Name	Set Reminders

Actors	User (Gardener), Notification Service (FCM/APNs/SendGrid/Twilio)
Preconditions	User authenticated. Notification permissions granted on device.
Trigger	User taps + Add Reminder on Reminders screen.
Basic Flow	1. User selects task type: Planting, Harvest, Fertilising, or Watering. 2. User sets date, time, and optional recurrence. 3. System presents Choose Notification Channel (Push/Email/SMS). 4. User selects channel. 5. System validates inputs and persists Reminder record (status=Pending). 6. System schedules notification via Notification Service. 7. Confirmation shown.
Alternative Flow	User cancels at any step: no reminder saved.
Post Conditions	Reminder record persisted. Notification scheduled.
Exceptions	Notification Service unavailable: Reminder saved; delivery retried when service restores (exponential backoff, max 5 retries).
Includes	Choose Notification Channel, Send Notification
Extends	Planting Reminder, Harvest Reminder, Fertilising Reminder, Watering Reminder

Field	Description
Use Case ID	UC-5.3
Name	Moderate Content
Actors	Admin/Moderator, User (Gardener)
Preconditions	Actor has Admin or Moderator role. At least one entry in the moderation queue.
Trigger	Moderator navigates to Moderation Queue, or a user report triggers a queue entry.
Basic Flow	1. Moderator opens queue and reviews the reported post or user profile. 2. Moderator selects action: Warn/Delete Post/Ban User. 3. System applies action. 4. System creates an immutable ModerationAction audit log entry. 5. System notifies affected user via their preferred channel. 6. Queue entry marked as resolved.
Alternative Flow	Moderator dismisses report as invalid: no action applied; report marked Reviewed with moderator note.
Post Conditions	Moderation action applied and audit-logged. Affected user notified.
Exceptions	Attempting to ban an already-banned user: system notifies moderator and offers unban instead.
Includes	Warn User, Delete Post, Ban User
Extends	Report Post/Report User (initiates queue entry)

3 Design Overview

GardenMate uses a layered service-oriented architecture following the COMET methodology: a React Native presentation layer on the client, a RESTful API Gateway, stateless application services for each subsystem, and a persistence layer combining PostgreSQL for relational data and MongoDB for document content. The five subsystems are independently deployable service modules.

3.1 Design Goals and Constraints

Methodology: All design artefacts conform to COMET (Gomaa, 2011). UML use case, sequence, class, and component diagrams are mandatory. The design follows a service-oriented incremental architecture. Platform: React Native targeting Android 8.0+ and iOS 14.0+. Platform-specific push notification code is isolated behind a NotificationAdapterInterface. Security: All REST communication encrypted with TLS 1.2+. Passwords stored as bcrypt hashes (cost factor >= 12). JWTs signed HS256; 24-hour access token TTL; 7-day refresh token TTL. OTPs are 6-digit, single-use, 5-minute TTL; account locked for 15 minutes after 5 failures. User location data stored with explicit consent (GDPR). All moderation actions written to an immutable audit log. Performance: Forum feed <= 3 seconds. Weather data <= 5 seconds. Recommendations <= 4 seconds. Push notifications <= 30 seconds of scheduled time. System sustains >= 500 concurrent users with <= 20% degradation.

3.2 Design Assumptions

Third-party API availability is governed by provider SLAs. GardenMate implements graceful degradation and caching to meet its own 99.5% uptime target. The device OS handles FCM/APNs push token registration; GardenMate stores the token against the user record. The Weather API returns JSON in the OpenWeatherMap schema; only the WeatherAdapter class requires change if a different provider is adopted. User devices have internet connectivity for core functionality; offline mode provides read-only access to cached plant data and forecast (up to 6 hours). React Native Expo managed workflow is used for v1.0.

3.3 Significant Design Packages

The GardenMate system is decomposed into the following design packages: (1) GardenMate-Client [Presentation]: React Native screens, navigation, Redux state management for all 5 subsystems. (2) API-Gateway [Integration]: Express.js gateway; TLS termination, JWT validation middleware, routing to subsystem services. (3) UserService-SS1 [Application]: Registration, OTP lifecycle, login, profile CRUD, preference management. (4) GuideService-SS2 [Application]: Crop database queries, recommendation engine, pest identification logic, guide content delivery. (5) WeatherService-SS3 [Application]: Weather API adapter, risk assessment, alert generation, protective action catalogue. (6) ReminderService-SS4 [Application]: Reminder CRUD, notification scheduling, snooze/reschedule, channel dispatch. (7) ForumService-SS5 [Application]: Post lifecycle, comment/like/save management, content filtering, moderation queue. (8) NotificationAdapter [Integration]: Unified interface for FCM, APNs, SendGrid, Twilio with exponential backoff retry. (9) WeatherAdapter [Integration]: OpenWeatherMap REST client with 30-minute cache. (10) PostgresDB [Persistence]: Users, reminders, posts, moderation log, OTP records. (11) MongoDB [Persistence]: Gardening guide content, crop data, pest catalogue, recommendation history.

3.4 Dependent External Interfaces

The table below lists the public interfaces this design requires from other modules or applications.

External Application / Interface	Module Using the Interface	Functionality / Description
OpenWeatherMap REST API	WeatherAdapter -> WeatherService (SS3), GuideService (SS2)	Provides current conditions and 5-day forecasts. Used for alert generation and planting recommendations. Cached 30 minutes. Uses GET /weather and /forecast endpoints.
Firebase Cloud Messaging (FCM)	NotificationAdapter -> ReminderService (SS4), UserService (SS1)	Android push notification delivery. OTP alerts and task reminders. Uses HTTP v1 API with device registration token.
Apple Push Notification service (APNs)	NotificationAdapter -> ReminderService (SS4), UserService (SS1)	iOS push notification delivery. OTP alerts and task reminders. Uses HTTP/2 protocol with certificate authentication.
SendGrid Email API	NotificationAdapter -> UserService (SS1), ReminderService (SS4)	Transactional email delivery for OTP codes and reminder emails. Uses POST /v3/mail/send endpoint.
Twilio SMS API	NotificationAdapter -> UserService (SS1), ReminderService (SS4)	SMS delivery for OTP codes and SMS-channel reminders. Uses POST /2010-04-01/Accounts/{SID}/Messages endpoint.

3.5 Implemented Application External Interfaces (and SOA web services)

The table below lists the implementation of public interfaces this design makes available for other applications.

External	Module Using the	Functionality / Description

Application / Interface	Interface	
POST /auth/register	UserService (SS1)	Accepts registration payload; triggers OTP dispatch; returns HTTP 201 on success.
POST /auth/login	UserService (SS1)	Validates credentials; returns JWT access and refresh tokens on success.
GET /recommendations	GuideService (SS2)	Returns personalised crop recommendations for the authenticated user profile.
GET /weather/forecast	WeatherService (SS3)	Returns 7-day forecast and active weather alerts for the user location.
POST /reminders	ReminderService (SS4)	Creates a new reminder, schedules notification; returns HTTP 201 with Reminder object.
PATCH /reminders/:id/snooze	ReminderService (SS4)	Snoozes a reminder and auto-reschedules to the next available slot.
GET /forum/posts	ForumService (SS5)	Returns paginated forum posts; supports ?search= and ?tag= query parameters.
POST /forum/posts	ForumService (SS5)	Creates a new post; applies content filter; returns HTTP 201.
POST /forum/moderate	ForumService (SS5)	Admin/Moderator only; applies warn/delete/ban action; writes audit log.

4 Logical View

This section describes the detailed design. The top-most layer shows how the five application service modules interact with the API Gateway and each other to complete the key use cases from Section 2. The service layer is then expanded to show how the significant classes within each module collaborate to implement the required behaviour. Each subsystem is a cohesive service module containing a Controller (REST handler), Service (business logic), Repository (data access), and domain-specific Engine or Adapter classes.

4.1 Design Model

The GardenMate design model uses a service-layer decomposition following COMET principles. The significant classes for each subsystem and their responsibilities are described below.

SS1 - UserService Classes:

AuthController: REST endpoint handler for /auth routes. Methods: register(req,res), login(req,res), verifyOTP(req,res), resetPassword(req,res).

AuthService: Business logic: credential validation, OTP lifecycle, JWT issuance. Methods: hashPassword(pw), validateOTP(code,userId), issueJWT(userId), refreshToken(token).

OTPGenerator: Generates cryptographically random 6-digit OTPs and sets TTL. Methods: generate(userId,channel):OTP, invalidate(otpId).

UserRepository: CRUD operations on PostgreSQL users table. Methods: create(dto), findByEmail(e), findById(id), updateStatus(id,status), updateProfile(id,dto).

ProfileController: REST handler for profile and preferences. Methods: getProfile(req,res), updateProfile(req,res), setPreferences(req,res), setLocation(req,res).

SS2 - GuideService Classes:

RecommendationController: REST handler for recommendation and guide endpoints. Methods: getRecommendations(req,res), searchCrops(req,res), getGuide(req,res).

RecommendationEngine: Correlates weather snapshot, soil type, and preferences against crop database to produce ranked crop list. Methods: generate(userProfile,weatherSnapshot):Recommendation[].
CropRepository: MongoDB CRUD for crop and guide content documents. Methods: findBySoilType(soil), searchByName(q), findAll().
PestController: REST handler for pest report submission and tip retrieval. Methods: reportPest(req,res), getTips(req,res).
PestIdentifier: Matches symptom description and optional image metadata to pest catalogue entries. Methods: identify(symptoms,imageMetadata):PestMatch[].

SS3 - WeatherService Classes:
WeatherController: REST handler for forecast and alert endpoints. Methods: getForecast(req,res), getAlerts(req,res).
WeatherAdapter: Adapter over OpenWeatherMap REST API; manages 30-minute cache. Methods: fetchCurrent(lat,lon), fetchForecast(lat,lon), getCached(key).
RiskAssessor: Evaluates weather data against configurable thresholds; produces WeatherAlert[]. Methods: assess(weatherData,thresholds):WeatherAlert[].
ActionAdvisor: Maps WeatherAlert type to a list of recommended protective gardening actions. Methods: getActions(alertType):string[].

SS4 - ReminderService Classes:
ReminderController: REST handler for CRUD and snooze operations. Methods: create(req,res), list(req,res), snooze(req,res), markDone(req,res).
ReminderScheduler: Calculates next fire time; registers with node-cron job queue. Methods: schedule(reminder), cancel(reminderId), reschedule(reminderId,newTime).
NotificationDispatcher: Selects channel adapter and dispatches notification message. Methods: dispatch(reminder):void, retry(jobId,attempt).
NotificationAdapter: Unified interface for FCM, APNs, SendGrid, Twilio with exponential backoff. Methods: sendPush(token,payload), sendEmail(to,subject,body), sendSMS(phone,text).
ReminderRepository: PostgreSQL CRUD for Reminder records. Methods: create(dto), findUpcoming(userId), update(id,dto), delete(id).

SS5 - ForumService Classes:
PostController: REST handler for post lifecycle. Methods: create(req,res), list(req,res), getById(req,res), like(req,res), save(req,res), report(req,res).
ContentFilter: Scans post body for malicious URLs and offensive keywords before persistence. Methods: validate(text):FilterResult.
CommentController: REST handler for comment CRUD on posts. Methods: addComment(req,res), listComments(req,res), deleteComment(req,res).
ModerationController: Admin/Moderator REST handler; enforces role guard. Methods: getQueue(req,res), applyAction(req,res).
ModerationService: Applies warn/delete/ban and writes immutable audit log. Methods: warn(targetUserId), deletePost(postId), banUser(userId), logAction(dto).
PostRepository: PostgreSQL CRUD for Post and Comment records. Methods: create(dto), findPaginated(page,limit,filter), findById(id), updateStatus(id,status).

4.2 Use Case Realization

For each use case defined in Section 2, the following describes how the scenario is realized within the GardenMate design, showing the sequence of interactions between components.

UC-1.1 Register Sequence:
1. Client: User taps Register; RegistrationForm rendered.
2. Client -> API Gateway: POST /auth/register {name, email, phone, password}.
3. API Gateway -> AuthController: Route dispatched; AuthController.register() invoked.
4. AuthController -> AuthService: authService.register(dto).
5. AuthService -> UserRepository: userRepo.findByEmail(email) - duplicate check.
6. AuthService: bcrypt.hash(password, 12) - credential hashing.
7. AuthService -> UserRepository: userRepo.create({...dto, passwordHash, status:'pending'}).

8. AuthService -> OTPGenerator: otp = otpGenerator.generate(userId, channel).
9. AuthService -> NotificationAdapter: notificationAdapter.sendEmail(email, otp.code).
10. AuthController -> Client: HTTP 202 {userId, message:'OTP sent'}.
11. Client: Renders OTPVerificationScreen.
12. Client -> API Gateway: POST /auth/verify-otp {userId, code}.
13. AuthService -> OTPGenerator: otpGenerator.validate(userId, code).
14. AuthService -> UserRepository: userRepo.updateStatus(userId, 'active').
15. AuthService: authService.issueJWT(userId) - generates access + refresh tokens.
16. AuthController -> Client: HTTP 200 {accessToken, refreshToken}.
17. Client: Stores tokens; navigates to Dashboard.

UC-3.2 Get Weather-Based Alerts Sequence:
1. Scheduler/Client: Trigger - 30-minute cron job or user opens Dashboard.
2. Client -> API Gateway: GET /weather/alerts?lat={lat}&lon={lon}.
3. API Gateway -> WeatherController: Route dispatched; WeatherController.getAlerts() invoked.
4. WeatherController -> WeatherAdapter: weatherAdapter.fetchCurrent(lat, lon).
5. WeatherAdapter: Check cache - if hit and age < 30 min, return cached data.
6. WeatherAdapter -> OpenWeatherMap: GET /weather?lat=...&lon=...&appid=... (on cache miss).
7. WeatherAdapter: Store response in cache with timestamp.
8. WeatherController -> RiskAssessor: riskAssessor.assess(weatherData, defaultThresholds).
9. RiskAssessor: Check thresholds - temp<3C: FrostAlert; temp>38C: HeatwaveAlert; rain>25mm/h: HeavyRainAlert; wind>60km/h: WindAlert.
10. WeatherController -> ActionAdvisor: actionAdvisor.getActions(alert.type) for each alert.
11. WeatherController -> Client: HTTP 200 {forecast, alerts:[{type, level, actions:[]}]}.
12. Client: Renders alert banners on Dashboard; dispatches push if app is backgrounded.

UC-4.1 Set Reminders Sequence:
1. Client: User taps + Add Reminder; ReminderCreateScreen renders.
2. Client: User selects task type, date/time, recurrence, channel (Push/Email/SMS).
3. Client -> API Gateway: POST /reminders {userId, type, scheduledTime, channel, recurrence}.
4. API Gateway -> ReminderController: ReminderController.create() invoked.
5. ReminderController -> ReminderRepository: reminderRepo.create({...dto, status:'pending'}).
6. ReminderController -> ReminderScheduler: reminderScheduler.schedule(reminder).
7. ReminderScheduler: Registers cron job to fire at scheduledTime.
8. ReminderController -> Client: HTTP 201 {reminder: {...}}.
9. Client: Appends reminder to upcoming list; shows confirmation toast.
10. [AT FIRE TIME] ReminderScheduler: Cron fires; calls notificationDispatcher.dispatch(reminder).
11. NotificationDispatcher -> NotificationAdapter: sendPush(token,payload) OR sendEmail(...) OR sendSMS(...).
12. NotificationDispatcher -> ReminderRepository: reminderRepo.update(id, {status:'fired', firedAt:now}).

UC-5.3 Moderate Content Sequence:
1. Moderator Client: Navigates to Moderation Queue screen.
2. Client -> API Gateway: GET /forum/moderate/queue (Authorization: Bearer {moderatorJWT}).
3. API Gateway: JWT middleware validates token; role check: must be 'admin' or 'moderator'.
4. ModerationController -> PostRepository: postRepo.findByStatus('reported').
5. ModerationController -> Client: HTTP 200 {queue: [ReportedItem[]]}.
6. Moderator: Reviews item and selects action: Warn/Delete Post/Ban User.
7. Client -> API Gateway: POST /forum/moderate {reportId, action, targetId, targetType}.
8. ModerationController -> ModerationService: moderationService.applyAction(dto).
9a. [If Delete] ModerationService -> PostRepository: postRepo.updateStatus(postId, 'deleted').
9b. [If Ban] ModerationService -> UserRepository: userRepo.updateStatus(userId, 'banned').
10. ModerationService -> ModerationRepository: moderationRepo.create({moderatorId, action, targetId, timestamp}).
11. ModerationService -> NotificationAdapter: Notify affected user via preferred channel.
12. ModerationController -> Client: HTTP 200 {success:true, actionApplied:action}.

5 Data View

This section describes the persistent data storage perspective of GardenMate. Relational entities (Users, OTPs, Reminders, Posts, Comments, ModerationActions) are stored in PostgreSQL. Document entities (Crops, GardeningGuide, PestCatalogue) are stored in MongoDB for flexible schema evolution.

5.1 Domain Model

Key domain relationships: User 1--* Reminder (a user owns many reminders). User 1--* Post (a user authors many posts). Post 1--* Comment (a post has many comments). ModerationAction *--1 User (many moderation actions may target a user; each performed by one moderator). Recommendation *--1 User (recommendation history tied to user profile). WeatherAlert is derived per request from WeatherData - not persisted beyond session cache.

5.2 Data Model (persistent data view)

5.2.1 Data Dictionary

User [PostgreSQL]: id (UUID PK), name, email (UNIQUE), phone (UNIQUE), passwordHash, locationLat, locationLon, soilType, organicMode (bool), waterSaving (bool), notificationChannel, status, createdAt. Status in {pending, active, locked, banned}. passwordHash uses bcrypt cost 12.
OTP [PostgreSQL]: id (UUID PK), userId (FK->User), code (6-digit), channel, expiresAt, used (bool), createdAt. Single use - used set true on validation. TTL = 5 minutes.
Reminder [PostgreSQL]: id (UUID PK), userId (FK->User), type, scheduledTime, channel, recurrence, status, firedAt, createdAt. type in {Planting, Harvest, Fertilising, Watering}. status in {pending, snoozed, done, missed, fired}. channel in {Push, Email, SMS}.
Post [PostgreSQL]: id (UUID PK), authorId (FK->User), title, body (TEXT), imageUrls (ARRAY), likeCount, status, createdAt, updatedAt. status in {published, reported, deleted}. ContentFilter applied before insert.
Comment [PostgreSQL]: id (UUID PK), postId (FK->Post), authorId (FK->User), body (TEXT), createdAt. Cascade delete when parent Post deleted.
ModerationAction [PostgreSQL]: id (UUID PK), moderatorId (FK->User), targetType, targetId, action, reason (TEXT), timestamp. Immutable - no UPDATE or DELETE permitted. action in {Warn, DeletePost, BanUser}. targetType in {Post, User}.
Crop [MongoDB]: _id, name, soilTypes[], minTemp, maxTemp, growthWeeks, wateringFrequency, companionPlants[], category. Collection: crops. Indexed on soilTypes and name.
GardeningGuide [MongoDB]: _id, category, title, content (rich text), tags[], updatedAt. Collection: guides. Full-text index on content for search.
PestCatalogue [MongoDB]: _id, name, symptoms[], affectedPlants[], organicTreatments[], chemicalTreatments[], severity. Collection: pests. severity in {low, medium, high}.

6 Exception Handling

GardenMate uses a centralised error handling middleware on the API Gateway layer. All services throw typed application exceptions that are caught and translated to standardised JSON error responses. All exceptions include a correlationId header (X-Correlation-ID) generated at the gateway, propagated through service calls, and included in all log entries. The defined exceptions are:

AuthenticationException [SS1]: Invalid credentials on login. Response: HTTP 401 {error:"InvalidCredentials"}. Increment failed attempt counter. Logged to security log.
OTPExpiredException [SS1]: OTP presented after 5-minute TTL. Response: HTTP 400 {error:"OTPExpired", message:"Request a new code"}. Not logged.
OTPLockoutException [SS1]: 5 consecutive failed OTP attempts. Response: HTTP 429 {error:"TooManyAttempts", retryAfter:900}. Account locked 15 minutes. Logged to security log.
DuplicateAccountException [SS1]: Registration with existing email/phone. Response: HTTP 409 {error:"AccountExists"}. Not logged.

WeatherAPIException [SS3]: OpenWeatherMap API unreachable or 5xx. Serve cached data if < 6 hours old; else return degraded response with warning flag. HTTP 200. Logged to ops log.
NotificationDeliveryException [SS4]: FCM/APNs/SendGrid/Twilio delivery failure. Retry with exponential backoff (delays: 30s, 2m, 5m, 15m, 30m). After 5 failures, mark job as failed and alert ops dashboard. Logged to delivery log.
ContentFilterException [SS5]: Post body fails content filter. Response: HTTP 422 {error:"ContentViolation", fields:["body"]}. Post not persisted. Logged to moderation log.
UnauthorizedModerationException [SS5]: Non-moderator attempts moderation endpoint. Response: HTTP 403 {error:"Forbidden"}. Logged to security log.
InvalidInputException [All]: Request body fails schema validation. Response: HTTP 400 {error:"ValidationError", fields:[{field,message}]}. Not logged.
ResourceNotFoundException [All]: Requested entity not found in DB. Response: HTTP 404 {error:"NotFound"}. Not logged.
InternalServerException [All]: Unhandled runtime error. Response: HTTP 500 {error:"InternalError"}. Stack trace suppressed in response; full trace logged to error log.

7 Configurable Parameters

The following parameters are externalised from code and managed via environment variables or a runtime configuration store. Dynamic parameters can be changed without restarting the application.

This table describes the simple configurable parameters (name / value pairs).

Configuration Parameter Name	Definition and Usage	Dynamic?
WEATHER_API_KEY	OpenWeatherMap API key. Injected at deploy time. Never committed to source control.	No
WEATHER_CACHE_TTL_MIN	Minutes to cache weather API responses. Default: 30. Lower values increase API quota usage.	Yes
WEATHER_REFRESH_INTERVAL_MIN	Interval for background weather refresh cron. Default: 30.	Yes
FROST_THRESHOLD_CELSIUS	Temperature (Celsius) at or below which a Frost Alert is generated. Default: 3.	Yes
HEATWAVE_THRESHOLD_CELSIUS	Temperature (Celsius) at or above which a Heatwave Alert is generated. Default: 38.	Yes
HEAVY_RAIN_THRESHOLD_MM_H	Precipitation rate (mm/h) triggering Heavy Rain Alert. Default: 25.	Yes
WIND_THRESHOLD_KMH	Wind speed (km/h) triggering Wind Alert. Default: 60.	Yes
JWT_SECRET	HS256 signing secret for JWT tokens. Rotated via secret manager.	Yes
JWT_ACCESS_TTL_H	JWT access token TTL in hours. Default: 24.	No
JWT_REFRESH_TTL_D	JWT refresh token TTL in days. Default: 7.	No
OTP_TTL_MIN	OTP time-to-live in minutes. Default: 5.	No
OTP_MAX_ATTEMPTS	Maximum failed OTP attempts before account lockout. Default: 5.	Yes
OTP_LOCKOUT_MIN	Duration (minutes) of OTP lockout after max failures. Default: 15.	Yes
BCRYPT_COST	bcrypt work factor for password hashing. Default: 12.	No
NOTIFICATION_RETRY_MAX	Maximum notification delivery retry attempts. Default: 5.	Yes
NOTIFICATION_RETRY_DELAYS_SEC	Comma-separated exponential backoff delays (seconds). Default: 30,120,300,900,1800.	Yes
FORUM_PAGE_SIZE	Default number of forum posts per paginated	Yes

	request. Default: 20.	
CONTENT_FILTER_ENABLED	Toggle content filtering on post creation. Default: true. Disable only for testing.	Yes
DB_POOL_MIN	Minimum PostgreSQL connection pool size. Default: 2.	No
DB_POOL_MAX	Maximum PostgreSQL connection pool size. Default: 20.	No

8 Quality of Service

This section describes aspects of the design related to application availability, security, performance, and monitoring and control in production.

8.1 Availability

GardenMate targets 99.5% monthly uptime (approximately 3.6 hours unplanned downtime per month). Design decisions supporting this: Stateless application services allow horizontal scaling behind a load balancer so any instance failure is transparent to clients. Graceful degradation - WeatherService returns cached data when external API is unavailable; ReminderService queues notifications locally and retries on service restoration. Database connections use a pool (min: 2, max: 20) with automatic reconnect. Planned maintenance uses rolling deploys with zero-downtime migration scripts. Health check endpoints (GET /health) are exposed for load balancer probes; an instance is removed if it fails 3 consecutive checks.

8.2 Security and Authorization

Authentication: All API requests except /auth/register and /auth/login require a valid JWT Bearer token. JWTs are signed with HS256; access tokens expire in 24 hours; refresh tokens in 7 days. Token revocation is managed via a Redis blocklist for logout. Passwords are stored as bcrypt hashes (cost 12) and never returned in any API response. Authorisation: Role-based access control with roles gardener (default), moderator, and admin. Role is embedded in the JWT payload and validated by middleware. Moderation endpoints require role in {moderator, admin} - HTTP 403 otherwise. Users may only modify their own profile and reminders; service layer enforces userId match. Transport and Data Security: All client-server communication uses TLS 1.2 minimum. Location data collected only with explicit opt-in consent (GDPR Article 6(1)(a)). Forum content undergoes server-side content filtering before persistence. All moderation actions written to an immutable audit log (INSERT-only on ModerationAction table).

8.3 Load and Performance Implications

Based on target user base of 1,000+ concurrent gardeners with growth to 5,000 within 12 months: Concurrent users (500+ with <= 20% degradation): horizontal auto-scaling; load balancer with stateless REST services. Weather API calls (~500 unique locations per 30 min): WeatherAdapter cache (TTL 30 min) aggregates requests by location to stay within rate limits. Forum feed load (<= 3 seconds): paginated queries (default 20 posts/page); indexed on (status, createdAt DESC); CDN for image serving. Recommendations (<= 4 seconds): RecommendationEngine offloaded to worker thread pool (Node worker_threads) to avoid blocking event loop. Database growth (Posts, ~50/day -> 18,000/year): archived posts older than 1 year moved to cold storage monthly; indexes reviewed quarterly. Notification throughput (~5,000 reminders/day peak): ReminderScheduler uses priority queue; NotificationAdapter batches FCM sends (up to 500/request via FCM multicast).

8.4 Monitoring and Control

Controllable Processes: ReminderScheduler (cron daemon) - start/stop via PM2 process manager; graceful shutdown drains in-flight job queue; health state exposed via GET /health/scheduler. WeatherRefreshJob (cron daemon) - polls on WEATHER_REFRESH_INTERVAL_MIN schedule; can be paused without affecting on-demand requests. NotificationRetryWorker - processes retry queue; pause/resume via admin API (POST /admin/notification-worker/{pause|resume}; requires admin JWT). Published Metrics:
gardenmate.api.request_count - total HTTP requests per endpoint per minute; alert if p95 latency > 5s.
gardenmate.api.error_rate - percentage of 4xx/5xx responses; alert threshold > 5% over 5-minute window.
gardenmate.weather.cache_hit_ratio - ratio of cache hits to total weather requests; low ratio triggers API quota

alert. gardenmate.notification.delivery_success_rate - percentage delivered on first attempt; alert if < 95%. gardenmate.notification.retry_queue_depth - jobs awaiting retry; alert if > 500. gardenmate.scheduler.jobs_fired - reminder notifications dispatched per hour. gardenmate.db.pool_utilisation - active connections/pool max; alert if > 80%. gardenmate.forum.moderation_queue_depth - unresolved reports; alert if > 50. All metrics published to a Prometheus-compatible endpoint (GET /metrics).